Inf2-SEPP 2025-26

Coursework 3

Software implementation of an events app

This coursework requires team work for SE (see 1), as well as working individually for ProP (see 2).

1 Your SE work (team work)

1.1 Update, March 20th

We provide the following clarifications/amendments, which are already included in the document but are centrally tracked here as well.

- The Cancel Performance sequence diagram incorrectly had a cancelByProvider() message passed to Event, instead of Booking. This has been fixed.
- The sequence diagrams contained a message checkEventIsTicketed() to Performance instead of a checkIfEventIsTicketed(). This has been fixed.
- Task 2 initially incorrectly made a reference to an EventPerformance class, which our diagram does not contain. It should have said Performance class. **Note that:** the classes mentioned in Task 2 refer to the original functionality as per our class diagram. If in your implementation, you decide to break the functionality of any of the classes mentioned in Task 2 into more than one class, you should provide unit tests for all such classes.
- The class diagram did not have a "Register the Enterteinmet Provider" option for the GuestMenu. This has been fixed.
- The Controller initially had private operations. This has been changed to protected.

- The MenuController incorrectly had a handlesTeachingStaff() operation; that has been fixed to handlesEnterntainmentProvider()
- The explanation for how the selectMenu() operation works has been updated to reflect its usage of getUserInput().

1.2 Introduction

The aim of this coursework is to implement and test the events app requested by the University of Hindeburgh, as well as reflect on professional issues related to it.

This coursework builds on Coursework 1 on requirements capture and Coursework 2 on design. Please refer back to the Coursework 1 and Coursework 2 instructions as needed.

You have access to the following documents as part of this Coursework:

1. Class diagrams covering most of the system
2. Sequence diagrams covering several important use cases
   As an alternative, all diagrams are available here as well. This should allow easier analysis.
3. A development tools document
4. A partial class diagram covering an experimental feature, **not part of the main system**.
5. Example of a system test. This is a system test based on the use case description of the "Request supermarket information" use case, extension 1a, from the first tutorial.
6. Mock API for external systems
7. SE marking scheme
8. ProP marking scheme

Important! The diagrams are based on the requirements from CW1 and CW2. Moreover, the design that you will work with in this coursework is an extension of the previous solutions (CW2 solutions launched on March 13th). However, *if there is any contradiction between the documentation provided in this coursework with anything from the previous courseworks and their solutions, you should follow the documentation from this coursework. You should also not at any point refer to your own requirements document from your CW1 submission or your own design document from your CW2 submission.*

1.2.1 Using approaches, tools and agile practices

This coursework is a small-scale opportunity to try out approaches and software tools that have been mentioned in the lectures starting with Week 6. For this coursework (only), please assume that you are using an *agile software development process*. As a result, you should pay attention to the *agile principles* from Lecture 2, and you are encouraged to use any applicable *practices* that have been proposed by the agile processes (described in Weeks 7, 8. and 9 of the course). You do not need to stick to one process, but you are free to mix and match practices from different processes that would make sense in your view for the system that you are developing and your team. Please be careful about practices that are dependent on one another! Furthermore, you are encouraged to try out software tools that are recommended for your chosen practices.

Apart from the software tools that are recommended for working in an agile process, we also recommend that you experiment with other tools that may be helpful for your development.

Overall, apart from the required tools mentioned in the next section, you could try out for example GitHub and the Git version control system [1], a test coverage tool like the default one provided by IntelliJ IDEA[2] or one of its alternatives (if you decide to use this IDE), a bug tracking tool like Trac[3] or JIRA[4], a static analysis tool like SpotBugs[5] or Infer[6], a project management tool like Trello[7], and/or any tools for support with agile like JIRA.

It is up to you how many and which approaches, tools and agile practices you use, and it should be based on your judgement of what could be beneficial for developing this system. The effort you put in will also influence the quality of the reflection which you make in SE Task 5 and your assessment for this section.

1.2.2 Server API

There are a number of external systems that our app would have to query or send information to at certain times, but your team does not have access to the code of those systems. In order to simulate this without having to deal with networking (which is a topic for another course), we will be using a mock API that specifies a communication contract between our system and the external ones.

In the same folder as this document, this API is provided in the 'external' folder. For example, `PaymentSystem` is an interface for the entertainment provider's verification system. Then `MockPaymentSystem` is a mock implementation of that interface.

[1] see the Git and GitHub tutorial from the Learn course under Other Resources
[2] `https://www.youtube.com/watch?v=QDFI19lj4OM`
[3] `https://trac.edgewall.org/`
[4] `https://www.atlassian.com/software/jira/bug-tracking`
[5] `https://spotbugs.github.io`
[6] `https://fbinfer.com`
[7] `https://trello.com/en-GB`

1.2.3 Pre-register Faculty Members Use Case—Alternative implementations

You will be asked to implement a new use case that is experimental in nature and not part of the main system. This use case can be implemented separately and without considering the rest of the system. This use case corresponds to pre-registering one more type of user in the system: faculty members. For this coursework, we will consider two alternative implementations of this use case. Different teams will implement **one** of them and will participate in code review along with another group with the other implementation (see Task 4 in section 1.3.4). We provide the part of class diagram corresponding to this use case in this link. The class diagram is partial and only provides rough guidelines for your implementation on purpose. Feel free to extend it as suitable for your solution. You can draw it for your own convenience, but you do not need to provide an updated class diagram as part of your submission.

You will eventually be given a file from the University containing login information for their faculty members (email addresses and encrypted passwords). For now, the University representatives want to see your implementation and tests of the use case before such file is provided. For the purpose of testing this use case and its classes, you can write your own mock file to simulate the file you would obtain from the university.

If your group number is *odd*: Your code should handle the new type of user and implement eager user migration: create user accounts for all faculty members in the file. Upon the first login, the user should be prompted to change the password if they wish.

If your group number is *even*: Your code should handle the new type of user for faculty members and implement lazy user migration: only upon the first login attempt of a user with an email you have in the file, will the user account be created.

Depending on your group number, you should only implement one of the above functionalities. Follow guidelines from this document (section 1.2.5) when you're implementing this functionality. As part of your code reviewing task, you will be asked to show the code corresponding to this use case to another team.

1.2.4 Development tools

Your company requires its developers to use certain development tools. We provide a Development Tools document that specifies these tools, their required versions, and how to install them (provided here). You will also need to know how to create and run tests using JUnit (version 5). The first thing is to set up your IDE with JUnit. If you go with our recommendation to use IntelliJ IDEA, here is a tutorial (go with the Maven option): `https://www.jetbrains.com/help/idea/junit.html`. Secondly, if you are not comfortable with using JUnit for testing, we suggest starting at the following links and doing more reading online: `https://www.vogella.com/tutorials/`

JUnit/article.html, https://bit.ly/3clJLs6, https://bit.ly/38t1ltn.

1.2.5 Following good practices in construction and testing

This coursework is a small-scale opportunity to try out ideas that have been mentioned in lectures. For example, you could try writing tests before code and using the tests to drive your design work. You could also try pair programming, whereby, in a session, one of you programs and the other gives feedback.

You are strongly encouraged to take an incremental approach, adding and testing features one by one as much as possible, always maintaining a system that passes all current tests.

Sometimes it is seen as important to have distinct development teams and testing teams. This way, there are two independent sets of eyes interpreting the requirements, and problems found during testing can highlight ambiguities in the requirements that need to be resolved. As you work through adding features to your design, you could alternate who writes the tests and who does the coding.

You may also find it useful to collaborate on your code via a shared git repository[8]. In bigger teams, it will likely be particularly important to do so.

Throughout this coursework, follow good coding practices as described in the lecture.You should attempt to follow the coding guidelines from Google at https://google.github.io/styleguide/javaguide.html. A common recommendation is that you never use tab characters of the default size set up by your IDE for indentation and you restrict line lengths to 80 or 100 characters. You are strongly recommended to adopt both of these recommendations because it is good practice and, particularly, in order to ensure maximum legibility of your code to the markers. A good practice is to adopt a consistent formatting style either via your IDE or a standalone tool (which can be used via an IDE) such as http://astyle.sourceforge.net/.

Be sure you are familiar with common interfaces in the Java collections framework such as List, Set, Map, and Queue/Deque and common implementations of these such as ArrayList, LinkedList, HashSet, TreeMap. Effective use of appropriate collection classes will help keep your implementation compact, straightforward and easy to maintain.

Browse the guidance at https://www.oracle.com/technetwork/java/javase/tech/index-137868.html on how to write Javadoc comments. Follow the guidance on including a summary sentence at the start of each comment separated by a blank line from the rest of the comment. This summary is used alone in parts of the documentation generated by Javadoc tools.

Use JUnit 5 to your advantage by using the right methods in your test class, the right annotations to test methods, and the right assertion methods (see Lecture 19). Your system test files will serve as your primary evidence and documentation for the correct

[8] https://guides.github.com/introduction/git-handbook/

functioning of your system. In terms of their documentation, take care with how you structure your test methods, give them intuitive names, add appropriate comments (for example, noting particular features being checked) and include a clear message when each test fails.

1.3 Tasks

There are several concurrent tasks you need to engage in. These tasks are provided in a certain order, but you may decide to change their order or do several of them in parallel, depending on your agile practices. These are described in the following subsections.

Bear in mind that it is not enough to submit code claiming to implement your system without testing and other documentation demonstrating its effectiveness. As part of this assignment, you are asked to implement tests demonstrating the correctness of some classes, and how your system implements the key use cases of the system, and **these tests will be used as the primary means of marking this coursework**.

1.3.1 Task 1: Construct code

Construct the code that implements the following use cases:

1. **Log in (for all users)**
2. **Log out (for all users)**
3. **Register entertainment provider**
4. **Create event**
5. **Search for performances**
6. **View performance**
7. **Book performance**
8. **Edit preferences**
9. **Cancel performance** (only for **groups of 3+**)
10. **Cancel booking** (only for **groups of 3+**)
11. **Review performance** (only for **groups of 4+**)
12. **Sponsor performance** (only for **groups of 4+**)

In doing so, make sure you closely follow the class diagram provided (will be released on Friday, March 13th) to you. Your code should ideally perfectly match the diagram. However, if you find that something from the provided class model cannot work in your implementation, or it can be designed in a better way, find a solution and justify in the text why you could not respect the class model. Moreover, say a few words about

how you would change the class model as a result to maintain consistency (check the submission section for the required format).

We assume that users will interact with your app with a text-based interface at this stage, so you do not need to worry about the User Interface (UI) beyond that.

When possible, ensure that your implementation fulfills desired quality attributes (e.g. security, privacy, reliability). Note that you need to reflect on this for Task 5.

Make sure to add comments to your methods. Leverage Javadoc and other types of comments to explain what the various parts of your code are doing, such that a colleague (or a marker) can understand it easily.

Include assertions in at least some of your methods as a means of catching invalid inputs, faults and invalid states.

Reaching an "Excellent" mark in this task

If you become confident that your solution to this task correctly respects all of its instructions from above and is of high quality, and you want to be able to reach a mark of "Excellent", you need to put in effort to go beyond what was required to impress the marker. The following, are some possible ways:

- Putting a lot of effort into checking for faults, invalid states, or doing input validation (reminder that this may lead to better security too, which you can use for your reflection) to the code; Briefly mention what you have done in the report.
- Related to the previous point, putting a lot of effort into quality (non-functional) attributes
- Frequently using well-known and high quality algorithms; Include their mention and a justification for using each in the report.
- Ensuring that your code is as free of "bad smells" as possible

This, together with excellent tests, can lead to exceptionality marks too.

1.3.2 Task 2: Unit testing

Use JUnit 5 to create unit tests for the following classes:

1. Performance
2. MockPaymentSystem (only for **groups of 3+**)
3. Booking (only for **groups of 4+**)

These tests should focus on checking whether the implementation of each specific class is correct. Ensure that all your tests pass. Place the tests for a class named MyClass in a file named TestMyClass.java, and use JUnit 5 assertions and annotations similarly to the examples provided in lectures and Tutorial 6. You don't need to test classic

getters/setters (this does not include getters and setters with more complicated functionality).

Note that for both this task and Task 3, since we assume student and admin staff are pre-registered, you can hardcode objects related to their accounts that you need for testing purposes.

In each test class, you should have several test methods for each method of the class that you are testing. Each test method should test for one combination of inputs as parameters. Make sure that you include frequent classes of inputs, but also more unusual ones, boundary and incorrect inputs for testing (see Tutorial 6 and its solutions for help). Each test method should be appropriately named such that the case you are testing is clear. Each test method should run the method that is being tested only once, i.e. you would normally have one single assert statement in each test method. Finally, include a clear message when the test fails.

Reaching an "Excellent" mark in this task

If you become confident that your solution to this task correctly respects all of its instructions from above and is of high quality, and you want to be able to reach a mark of "Excellent", you can try to test for all classes of inputs for each method and also use a tool to calculate statement and branch coverage in each of the tested classes and improve your code and retest until they are at least 70% (see Lecture 19). This, together with excellent code and system tests, can lead to exceptionality marks too.

1.3.3 Task 3: Create system-level tests

Your are expected to use JUnit 5 to create system-level tests that simulate the scenarios of each of the use cases you implemented in the last task (still respecting group sizes). Note that when you are testing all student- and admin-related use cases, you can assume you have pre-registered accounts and, for example, hard-code those accounts in your tests.

To get a good mark for this coursework you must test all the use cases you implemented since your tests will provide the main means for assessing your implementation. Even if you have implemented a feature, you may not get full marks if you have not tested your system sufficiently to demonstrate your system successfully implements the feature.

You are expected to add enough systems tests to completely test the relevant use cases.

Some advice

Few tests are able to test single use cases by themselves. In nearly all cases, several use cases are needed to set up some state, followed by one or more to observe the state. For example, you will need to register an entertainment provider, and have that user create an event and at least one performance for it, before the student can book that

performance. Additionally, you may want to include tests which perform more than one use case in turn to check the integration of the modules of the system.

Do not run all your tests together in one large single test. Where possible, check distinct features in distinct tests. However, don't rely on the order of the tests as Java does not necessarily run them in that order.

You are encouraged to adopt a test first approach — the tests are just as important a part of the assignment as is your implementation itself. Make use of the provided tests to help you develop your code, and when tackling some feature not already tested for, write a test that exercises it before writing the code itself.

Include all your system tests for each use case as JUnit test methods in a `[use-case-name] SystemTests.java` class (e.g. `LogInSystemTests.java`, `RegisterEntertainmentProvideSystemTests.java` etc.). Overall, you should have one such class for each use case. Your system test files will serve as your primary evidence and documentation for the correct functioning of your system. To this end, take care with how you structure your test methods, give them intuitive names, add appropriate comments (for example noting particular design features being checked) and include a clear message when each test fails.

To simulate user input for system testing, you can add Mockito `https://github.com/mockito/mockito` to your project, or google "Java JUnit simulate user keyboard input" to explore other alternatives. Follow a similar process to the one described in 1.2.4

Reaching an "Excellent" mark in this task

If you become confident that your solution to this task correctly respects all of its instructions from above and is of high quality, and you want to be able to reach a mark of "Excellent", you can try to cover through your system tests most if not all of the different scenarios for each case that you are tackling. This, together with excellent code and unit tests, can lead to exceptionality marks too.

1.3.4 Task 4: Code review

This task is about giving and getting feedback on a small part of your code in the form of a *code review* - doing this is a useful skill for a software engineer.

You will implement a new use case—along with its tests—that hasn't been a part of the courseworks so far. The use case is "Pre-register Faculty Members"(section 1.2.3). **Note that this task is separate from the rest of the coursework - the system you build for Task 1 should not include this "Pre-register Faculty Members" use case.**

Please join one of the Week 10 or 11 labs as a team, having prepared in advance the Pre-register Faculty Member implementation and its tests (system and unit tests). There, one of the lab demonstrators will first ensure that you have the necessary code and that it passes its tests (preconditions for the code review). Then, the demonstrator

will pair you up with another team and you will be asked to exchange with them your implementation and its system tests (and only them), and work separately to assess each other's work on them and write your notes in a text file, for 20 minutes. At the end of your allocated time, you will be asked to exchange your notes with the other team, and also submit them on Learn, all under the supervision of the demonstrator.

You will be provided with some guidance on what constitutes a good code review, and where you can submit your code review on Learn, in due course through a Learn announcement.

1.3.5 Task 5: Reflection (different from CW2)

This section asks you to reflect and self-assess your team's progress with this coursework. This is a great opportunity to take the time to learn from your experience with this coursework, and thus develop your analytical thinking skills. It will also help you judge your work before we do, by interacting with the marking scheme and thus gaining an understanding of how you are being marked.

Before you start doing any reflection, here is a useful model (adapted from the Integrated Reflective Cycle (Bassot, 2013)) that you could use to structure your reflection:

1. **The Experience**: Describe what you did, what you tried out.
2. **Reflection on Action**: What were the results? What went well? What didn't? Why?
3. **Theory**: What have you learned from this experience?
4. **Preparation**: What could you have done to make things better, according to the lessons learned? If you have the chance to do this again (e.g. team work), what will you do or try out next time to try to make things better?

You can see an example of this model being put to use at this link.

1) Reflection on teamwork

Using the above reflection model, write one or two paragraphs summing up to *maximum 250 words* of reflection on your team's teamwork for this coursework. Focus on things such as how you got organised, split up responsibilities between team members, communicated, and managed progress in working towards the deadline for this coursework. Make sure to mention and reflect on the use of and usefulness of any tools that you tried out in this process, e.g. physical or online tools for managing your teamwork. You can use the reflection model repeatedly to describe how you improved your teamwork over time.

2) NEW! Reflection on the quality of your work and quality attributes of your work

Using the same reflection model, write one or two paragraphs summing up to *maximum*

250 words of reflection on the quality of your work. Mention how well you think you tackled the work in the different tasks. Touch upon one of the quality attributes (security, privacy, reliability etc.) your final system has. Show specific implementation or testing features that demonstrate the presence quality attributes. Reflect if such a quality attribute could have been achieved in a more efficient way/different way or on the changes you would have made if you had more time/resources. We recommend you have a look at the marking scheme from this link, making specific reference to parts of it, to help you structure this response, however touching on all marking criteria or marking yourself using it is not expected. You can use the reflection model repeatedly to describe how you improved your solutions over time.

3) NEW! Reflection on tools, agile approaches and practices (can lead to exceptionality marks) Using the same reflection model, write one or two paragraphs summing up to maximum 250 words of reflection on your use of tools, agile approaches and practices for this coursework (see 1.2.1). In particular, here it is useful to reflect on your reasoning for choosing each approach/tool/practice (and not choosing an alternative), how the approach/tool/practice worked for you, what you would conclude are its advantages/disadvantages, what you would do differently next time you develop a system. **IMPORTANT!** In your description, please include some screenshots of your use of any tools that you mention.

Important! You should make a real effort to be reflective, as well as honest, in this task. Please note that only making bold statements like "We did this excellently well", with no justification, and (even worse!) not being open to consider that there is always room for improvement, will result in very little, or even no, credit for this part.

1.3.6 Task 6: Declaration of work

You are also required to declare the amount of work [9] that was carried out by each of your team members, so that we can adjust your group mark accordingly. In particular, you are each expected to split up work *fairly*. This means each team member doing around:

- **Groups of 2**: half (50%) of the work
- **Groups of 3**: a third (33%) of the work
- **Groups of 4**: a quarter (25%) of the work

on this assignment (no matter how you split responsibilities), or to have agreed with teammates how to average out across courseworks in the case of personal circumstances.

[9] By "work" we mean a product of the effort and time spent throughout this coursework. Work contribution should thus not only include the contribution to the final submitted solutions, but also things like studying the theory, learning about teamwork, organising and attending team meetings, checking and providing feedback/improving others' solutions.

For this task, prepare the following:

1. As a team, a **Team Work Declaration document** (see submission details in Section 4) and do either a) or b) from below:

   a) If you consider *your work was split fairly according to the definition from the footer*, include in it the text "The work was split fairly between our team members". Then, each team member signs, *but only if they agree with the percentage next to their name* (see Terms below).

   b) If not, write for each team member an estimate (in percentages) of how much work they have carried out. Then, each team member signs, *but only if they agree with the percentage next to their name* (see Terms below).

   For the signature, a digital one or a picture of one taken with a mobile phone are both accepted. Alternatively, the document itself could be a picture of a handwritten document including the different team members' signatures.

2. Privately as an individual, fill in **the Individual Work Declaration Form from this link** to describe in your own words the work that you have put into this coursework. This will help us make a more informed decision regarding your mark.

Terms: You have 30 extra minutes to submit the two declarations. Failure to submit the Team Work Declaration document (even in the presence of submitted Individual Work Declaration Forms), or submitting it after the deadline + 30 minutes, will lead to our assumption that you have split the work fairly, and you will all receive the same mark without any objections possible. We will accept a Team Work Declaration document without all signatures included, as long as it is submitted within 30 minutes after the deadline. However, note that submitting it without all signatures will lead to a potentially difficult conversation between the course organiser and the team members. In the absence of any submitted Individual Work Declaration Forms, a fully signed Team Work Declaration document option a) will result in the same mark for all the team members, while option b) will mean bringing the mark down for anybody with a lower percentage than expected, in accordance with the declared percentage (e.g. if a team member in a 4-person team did 15% of the work, this is 60% of the 25% they needed to do, so they will receive 60% of the team's mark). Individuals who have done more than their share will not have their mark increased according to the percentage, but if the criteria for excellence and exceptionality for a very high mark are met (see marking scheme), they may be considered for a small bonus (at the course organiser's discretion) after all courseworks. When Individual Work Declaration Forms are also submitted within 30 minutes after the deadline, they will also be used to decide on the individual marks, with the same considerations for students putting in more than their share of the work as above.

2 Your ProP Work (individual)

As part of CW3 (ProP) you are required to write an essay choosing ***one of the topics below***.

The length of the essay should be ***around 750 words and maximum 1000 words***. The essay should be ***argumentative in nature*** (i.e., at its core, it should argue for or against something, using premises and a conclusion). As such, it should also include at least one counter-premise and its refutation. Please revisit Tutorial 1 and its solutions if you need help with the essay's structure.

There are no "right" answers–the main goal is to see your thoughts and reasoning based on evidence.

2.1 Topics (choose one)

Topic 1. Assume that the university also asked Acme Corp to include a map viewer with navigation in the events app, to help students get to events. As a software engineer working on the events app project, advocate for the adoption of a certain map library by Acme Corp. In doing so, compare this library with others and discuss the pros and cons/risks of going for a proprietary versus open source library.

Topic 2. Construct an argument supporting the actions that you should take as a software engineer at Acme Corp for ensuring data privacy and protection in the development of the university events app. Make sure you mention the types of data that you are referring to, and the risks associated with inadequate data privacy and protection.

Topic 3. Construct an argument supporting the approach that you should take as a software engineer at Acme Corp for addressing one or more applicable software engineering standards that are applicable to your university events app. In doing so, discuss the suitability of this approach and why it is more suitable than alternative approaches.

For all essays: connect your argument with at least one of the principles in the ACM Code of Ethics and Professional Conduct.

2.2 Writing the essay

When writing the essay, think about the assessment criteria for this ProP task– available on Learn as a marking scheme. Read carefully the criteria definitions and use them to guide your writing.

Here is a possible approach for writing your essay:

1. **Revisit relevant (course) materials**. Revisit (ProP) lectures 3, 12, 16, 21, 24, their associated reading and other useful sources (e.g., the first chapter of "A rulebook for arguments"). Carefully read the coursework description. You may also want to revisit relevant SE lectures.

2. **Brainstorm**. Take some time to write down your premises, conclusion, potential evidence. Write down ideas that support your conclusion but also opposing views. Think about various perspectives and write down ideas. Do not worry about your writing style or structure/organisation of ideas, just write ideas down, preferably using bullet points.
3. **Prepare**. Collect all the evidence and write down an outline of your essay. Take enough time to gather the evidence you need and check if it was reliable. This is the phase when you organise your ideas written down in phase 2 and think about how to unfold them (e.g., in which order).
4. **Draft**. Write a rough draft of your essay. Don't spend time checking how correct your sentences are. Just write and try to include any data/direct quotes as early as possible.
5. **Revise**. Polish your rough draft, optimise word choice, make sure your sentences are concise, and restructure your arguments if necessary. Make sure your language is clear, and double-check that you effectively made all your points and rebuttals. Also, remove redundant sentences and sentences which do not bring any value to your argument. *Do not expect to write your essay in one go, but develop it iteratively.*
6. **Proofread**. Read through your essay and focus exclusively on fixing mistakes. Use Grammarly to identify and fix grammatical errors.

2.3 Assessment for the ProP Tasks

Please find here a marking scheme for ProP.

3 Some advice (same as for CW1 and CW2)

3.1 Working as a team

To get organised and work effectively as a team, carry out the teamwork acivities in meetings and labs, have a look at the **Teamwork Resources** from here, and follow any advice on teamwork from guest lectures. Mention what you have done and used for teamwork in Task 6. If you need advice on teamwork, or you are experiencing difficulties with it which you have attempted to address for 7 days with no/little success, please ask for help (privately) on Piazza (tag with *teamwork*) or email your course organiser. She will discuss the matter privately with you and/or your peers, as appropriate.

3.2 Asking questions

Please ask questions in labs or on Piazza if you are unclear about any aspect of the system description or tasks. On Piazza, tag your questions using the *cw3* folder for this

coursework. As questions and answers build up on the forum, remember to check the existing questions first: maybe your question has already been answered!

In case of questions regarding the organisation of teamwork in this course, please consult the **Teamwork FAQ** document from this link . Use Piazza (tag with *teamwork*) or email your course organiser in case your question was not covered there

3.3 Good Scholarly Practice and Respecting the Law

Please remember the university requirement as regards all assessed work, outlined at this link. **This coursework allows AI-Assisted Planning only** (see the same link).

We will run a plagiarism and Generative AI checker on your solutions, and report you in case we suspect academic misconduct.

You are also required to take reasonable measures to protect your assessed work from unauthorised access. E.g., if you put any such work on an online repository like GitHub then you must set access permissions to allow access only to you and your team.

4 Submission

Please make **six** submissions for this coursework (**IMPORTANT!** Pay attention to whether the submission is individual or group):

- **As a group** SE tasks solutions and documents:
  1. A .ZIP file of your implementation, including all your tests and Javadoc as well (DO NOT include the implementation of the "Pre-register faculty member" use case here). To do this in IntelliJ, go to:

     `File - Export - Project to ZIP file`.

     Rename this file **CW3SEImplementationGroupX.zip**.
  2. A PDF (not a Word or OpenOffice document) of your report. The document should be named **CW3SEReportGroupX.pdf**, where you replace the X with your group number. Please **DO NOT include the names and/or UUNs of the team members, or the declaration of work** within this document, so that we can mark anonymously. This submission should be made by one person and is used for the whole group. Here you include any textual information to describe your choices as per Task 1 and your reflection (Task 5).
  3. A .ZIP file of your implementation and tests for your "Pre-register faculty member" use case. Name this file **CW3SECodeReviewImplementationGroupX.zip**.

4. The SE team declaration of work. This should include a PDF (not a Word or OpenOffice document) of your 'Declaration of work', entitled **CW3SEDeclarationGroupX.pdf**, where you replace the X with your group number. This document will only be accessed by your course organiser. This submission should also only be made by one of the team members, but this team member can be different to the one who submitted the requirements document.

- **As an individual:**
  1. The responses to the **Individual Work Declaration Form** of each team member. These should be filled in and submitted privately by each individual.
  2. Your **individual ProP Essay**. This should include a PDF (not a Word or OpenOffice document) of your report with the essay, named **CW3ProPreport.pdf**. Please **DO NOT include your name, or the declaration of work** within this document, so that we can mark anonymously.

How to Submit

Ensure you are logged into MyEd. Access the Learn page for the Inf2-SEPP course, go to "Assessment/Assignment 3: Construction". Except for the Individual Work Declaration, each submission will have its own submission box (to be opened 2-3 days before the deadline).

Submission is a two-step process for each: upload the file, and then submit. This will submit the assignment and receipt will appear at the top of the screen meaning the submission has been successful. The unique id number which acts as proof of the submission will also be emailed to you. **Please check your email to ensure you have received confirmation of your submission**.

If you do have a problem submitting your assignment try these troubleshooting steps:

- If it will not upload, try logging out of Learn / MyEd completely and closing your browser. If possible, try using a different browser.
- If you do not receive the expected confirmation of submission, try submitting again.
- If you cannot resubmit, contact the course organiser at asejfia@ed.ac.uk attaching your assignment, and if possible a screenshot of any error message which you may have.
- If you have a technical problem, contact the IS helpline (is.helpline@ed.ac.uk). Note the course name, type of computer, browser and connection you are using, and where possible take a screenshot of any error message you have.
- Always allow yourself time to contact helpline / your tutors if you have a problem submitting your assignment.

5 Deadline

Please submit the report for a final mark and feedback by:

12:00 (noon), Monday, 6th of April, 2026.

You are allowed to submit the team and individual Declarations of Work 30 minutes later. This coursework is worth 63% of the total coursework mark: 38% for the SE tasks, and 25% for the ProP tasks. We estimate it should take each team member around 26 hours of work (20 for SE tasks, 6 for ProP task).

Important! Extension rule 2 from this link is used for this coursework—see details for group submissions. This coursework allows AI-Assisted Planning only, as described at this link.

Borislav Ikonomov, Cristina Alexandru, Adriana Sejfia 2026.